Τμήμα Μηχανικών Η/Υ και Πληροφορικής

Υλοποιητικό Project - Ανάκτησης Δεδομένων

Χειμερινό εξάμηνο 2025

Λουκανάρης Κωνσταντίνος: 1100610, up1100610@ac.upatras.gr
Ραβασόπουλος Γιάννης: 1100696, up1100696@ac.upatras.gr

Table of Contents

Table of Contents **2**
Εισαγωγή: **3**
Θεωρία και Μοντέλα: 3
Υλοποίηση **4**
Ερώτημα 1 - Αναγνωση και Επεξεργασία Συλλογής 5
Ερώτημα 2 - Υλοποίηση Μοντέλου Διανυσματικού Χώρου 6
Αξιολόγηση ομοιότητας 7
TF - IDF και Weighting 7
Ερώτημα 3 - Υλοποίηση Μετρικών Αξιολόγησης Α.Π 8
1. Ακρίβεια - Precision 8
2. Ανάκληση - Recall 8
3. F1-Score - Αρμονικός Μέσος Όρος 8
4. Precision@k - Ακρίβεια στα k docs 9
5. Διάγραμμα Ανάκλησης-Ακρίβειας - Precision-Recall Curve 9
Ερώτημα 4 - Συγκρίσεις 10
Κριτήρια επιλογής βέλτιστου μοντέλου 10
1. MAP(Mean Average Precision): 10
2. P@10: 11
Ερώτημα 5 - Clustering με TF-IDF και προεκπαιδευμένες ενσωματώσεις 12
Αποτελέσματα και Παρατηρήσεις: **14**
Πειραματικό πλαίσιο: 14
Ερωτήματα 1:4 - Μετρικές και Αξιολόγηση 14
VSM Plots and Comparisons 15
Clustering Plots and Comparisons 17
Αναφορές - Βιβλιογραφία **19**
[2] Disproportionate Scaling in Normalization and effect on scale sensitive methods 19
Παράρτημα **20**

Εισαγωγή:

Θεωρία και Μοντέλα:

1. **Inverted Index:**
    a. *Introduction to Modern Information Retrieval, Gerard Salton, Michael J. McGill*

2. **Vector Space Modeling:**
    a. *Matrices, Vector Spaces, andInformation Retrieval - Michael W. Berry, Zlatko Drmac Elizabeth R. Jessup*

3. **BERT:**
    a. https://github.com/google-research/bert,
    b. https://huggingface.co/docs/transformers/en/model_doc/bert

Η υλοποίηση της παρακάτω εργασίας έγινε σε Python αξιοποιώντας τις παρακάτω βιβλιοθήκες:

- pandas
- matplotlib
- scikit-learn
- sentence-transformers
- numpy
- torch
- os
- math
- string
- time
- itertools

Για την εκτέλεση του κώδικα αρχικά:

- φτιάχνουμε ένα virtual environment (python3 -m venv .venv)
- το ενεργοποιούμε (source .venv/bin/activate)
- πλέον μέσα στο venv τρέχουμε `pip install torch --index-url https://download.pytorch.org/whl/cpu`
  αν θέλουμε να κατεβάσουμε το light pytorch που αξιοποιεί μόνο την CPU, χωρίς CUDA libraries etc. π.χ. όπως θα κάναμε σε ένα laptop (γιατί αυτό είναι 180MB~ αντί για 4GB), αλλιώς απλά κατεβάζουμε μόνο το requirements.txt που με τη σειρά του κατεβάζει τα needed libraries και το μεγάλο μοντέλο.
- τρέχουμε όπως και να έχει `pip install -r requirements.txt` για να κατεβάσουμε και τις βιβλιοθήκες και αν έχουμε κατεβάσει το light pytorch τότε το pip έξυπνα αγνοεί το “βαρύ” μέσα στο κατέβασμα του “sentence-transformers”.

Execution και συσχέτιση με τα ερωτήματα:

- τρέχοντας το main.py (python3 main.py) στην ουσία τρέχουμε τον κώδικα των ερωτημάτων 1, 2 και 3 , καθαρίζοντας τα αρχεία, φτιάχνοντας το inverted-index και μοντελοποιώντας τα έγγραφα μέσω VSM και έπειτα εκτύπωση και υπολογισμός παραμέτρων

- τρέχοντας το optimization.py (python3 optimization.py) εκτελούμε τον κώδικα που αναφέρεται στο ερώτημα 4, κάνοντας χρήση της scikit-learn και υπολογίζοντας τις καλύτερες παραμέτρους του TfidfVectorizer μαζί με άλλα.

- τέλος τρέχοντας το clustering.py (python3 clustering.py) εκτελούμε τον κώδικα που αναφέρεται στο ερώτημα 5 και με τα πλέον hardcoded best parameters του Vectorizer συγκρίνουμε τα clusters αυτού με του μοντέλου από το hugging face.

Υλοποίηση

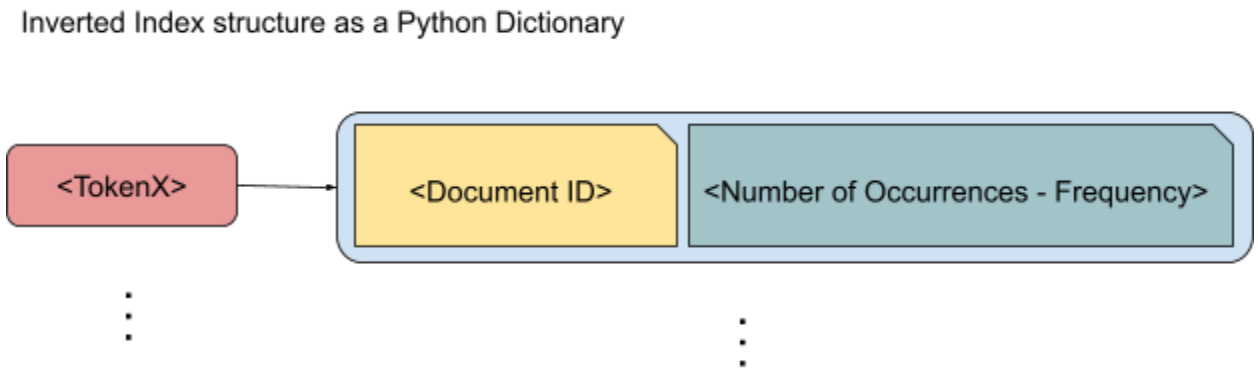
Ερώτημα 1 - Αναγνωση και Επεξεργασία Συλλογής

Στο στάδιο αυτό επιλέξαμε να υλοποιήσουμε την ανάγνωση των αρχείων μέσω της βιβλιοθήκης pandas για να μας προσφέρει εύκολη οργάνωση της πληροφορίας μέσω του dataframe format. Σε αυτή τη φάση το dataframe περιέχει τα document ids ευθυγραμμισμένα με τα περιεχόμενα τους.

Έπειτα για τη κατασκευή του inverted index προ-επεξεργαζόμαστε τα δεδομένα για την αφαίρεση stopwords και ένα στοιχειώδες-trivial stemming. Πλέον το dataframe έχει το νέο δεδομένο των “tokens” στο οποίο για κάθε id και αντίστοιχο content αποθηκεύεται και μία λίστα με τα separated και “καθαρισμένα” tokens που περιέχονται στο εν’ λόγω content. Η επιλογή του preprocessing είναι προφανώς το γεγονός πως σκοπεύοντας να αξιοποιήσουμε τα δεδομένα οργανώνοντας τα σε κάποιο vector space πρέπει να αποφύγουμε παραπλανητικά δεδομένα, όπως τα stopwords τα οποία δεν είναι αντιπροσωπευτικά του περιεχομένου - πληροφορίας ενός document αφου είναι λέξεις που συναντώνται συχνά σε πολλά αρχεία ανεξαρτήτως ομοιότητας λεκτικής ή και σημασιολογικής. Ιδιαίτερα η συμπερίληψή τους θα επέτρεπε σε document representation vectors στο vsm να εμφανίζονται skewed εμφανίζοντας παραπλανητικά trends και μην επιτρέποντας στην αναζήτηση μας να αποφανθεί ομοιότητας στο πλαίσιο της εργασίας.

Αντιλαμβανόμαστε οτι σε ιδιαίτερα σενάρια η συχνότητα εμφάνισης των stopwords είναι σημαντικό metric για την αξιοπιστία, τυπικότητα, επισημο ύφος κλπ. Στα πλαίσια της εργασίας κρίναμε σημαντικότερο να θυσιάσουμε αυτή τη πληροφορία προκειμένου να εξυπηρετηθεί ιδανικότερα το σενάριο αναζήτησης μας και όχι ενα γενικο vector space model για οποιοδήποτε είδους query.

Το inverted index μας αναπαριστάται από ένα python dictionary. Περιέχει ως κλειδί το αντίστοιχο token - word - term και ώς content το tuple document id και frequency. Η παραπάνω περιγραφή είναι η τυπική δομή ενός inverted index στη βιβλιογραφία. Η επιλογή του dictionary εναντι ενός list είναι η αποφυγή εξάρτησης από indices και πιθανά misalignments που θα μπορούσαν να προκύψουν αν αναπαριστούσαμε document ids μέσω του index μέσω του set το οποίο έχει κενά documents.



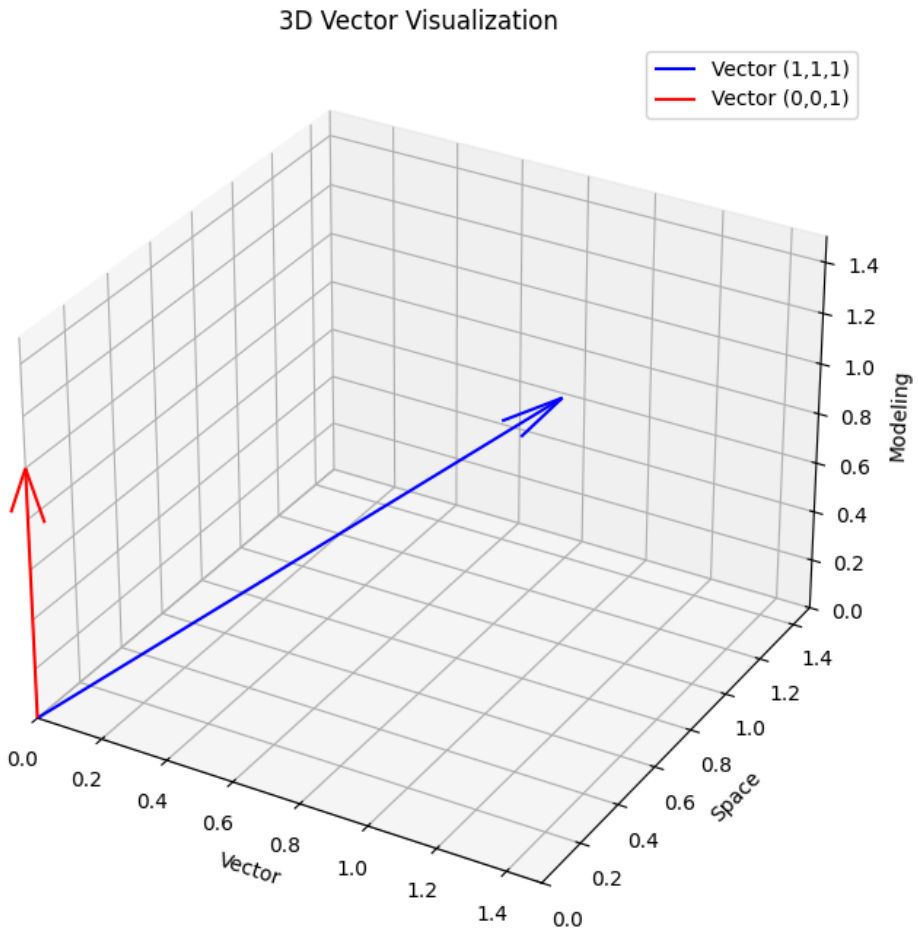
Ερώτημα 2 - Υλοποίηση Μοντέλου Διανυσματικού Χώρου

Διαισθητικά, στο χώρο που ορίζουν άξονες που αναπαριστούν όλα τα πιθανά tokens του συνολικού corpus, κάθε tuple εγγραφής του dictionary αναπαριστά ένα document του οποίου οι επιμέρους συνιστώσες βαθμονομούνται κατα το πλήθος επαναλήψεων του αντίστοιχου token component.

Παρακάτω ένα απλοποιημένο σενάριο στο οποίο το corpus είναι τα documents:

- **D1: “Vector Space Modeling”**
- **D2: “Modeling”**

Ο χώρος μας έχει ώς βάση τα unit vectors “Vector”, “Space” και “Modeling”. Το μπλε διάνυσμα αναπαριστά το document 1, ενώ το κόκκινο: το document 2.



Αξιολόγηση ομοιότητας

Η αναζήτηση αφορά την συμπερίληψη ενός query vector στο static vsm και την αναζήτηση του καλύτερου candidate που ικανοποιεί το κριτήριο ομοιότητας. Για να αποφανθούμε της ομοιότητας αξιολογούμε τον βαθμό παραλληλίας και μήκους μεταξύ του query vector και όλων των document vectors στο σύστημα μας. Η ιδανική λύση που προσφέρει το vector space modeling για τέτοιου είδους αναζητησεις είναι το cosine similarity, η αξιολόγηση δηλαδή των τιμών αυτών μέσω του παρακάτω formula:

$$cos(\theta) = \frac{A * B}{||A||\,||B||}$$

TF - IDF και Weighting

Το ζύγισμα των components εξαρτάται απο την σημαντικότητα - περιγραφικότητα κάθε token στο corpus. Λέξεις που γενικά είναι φαινομενικά ασήμαντες αφαιρούνται στο preprocessing ώς stopwords. Το επόμενο φίλτρο είναι το punishing ή rewarding ζύγισμα των tokens ανάλογα με το τρόπο που θα αξιοποιηθεί το σύστημα ή ανάλογα με το query search space. Στα πλαίσια της εργασίας εκτελούμε ένα στατικό ζύγισμα με τη μορφή του TF-IDF και το alternative logarithmic TF-IDF που κανει nullify fluctuations και unclear trends αλλα διατηρεί το σχετικό magnitude info στα vectors του vsm.

$$idf = \frac{number\ of\ documents\ in\ corpus}{number\ of\ documents\ containing\ said\ token}$$

Αξιοσημείωτο είναι πως η εφαρμογή των βαρών είτε της κλασικής ή της alternative τεχνικής ζυγίσματος των components γίνεται κατα την αναζήτηση και όχι κατα τη κατασκευή του vsm. Αυτό μπορεί να καθυστερεί φαινομενικά την αναζήτηση αλλα αποτελεί καλύτερη τεχνική για accomodation δυναμικών τεχνικών αναζήτησης όπως bm25 κλπ.

Ζήτημα που μας προβλημάτισε επίσης αποτέλεσε το normalization ή Max Normalization του vectorspace και κατ' επέκταση του query vector για το optimization της αναπαράστασης χωρικά αλλα και επιτάχυνση των πράξεων για τον υπολογισμό του cosine similarity grade. Τελικά καταλήξαμε πως για να μην επηρεαστούν ούτε τα σχετικά relationships μεταξύ components κάνοντας scale ανα component max αδικώντας components με μεγάλο frequency αλλα και για να μην κάνουμε κάθε διάνυσμα μοναδιαίο χάνοντας το magnitude factor του αρα και το ενδεικτικό του skew προς οποιαδήποτε tokens είναι πιο information dense για το συγκεκριμένο document vector, να μην πειράξουμε καθόλου τη δομή του vsm και να εκτελέσουμε την κλασική μορφή των υπολογισμών ανα το search. [2]

Ερώτημα 3 - Υλοποίηση Μετρικών Αξιολόγησης Α.Π.

1. Ακρίβεια - Precision

Η ακρίβεια στην ουσία απαντά στο ερώτημα του “από όλα τα έγγραφα που επέστρεψε το σύστημα Α.Π. μας, πόσα είναι σχετικά;”.

Precision = $\frac{|\,Retrieved \cap Relevant\,|}{|\,Retrieved\,|}$, όπου | . | ο πληθάριθμος του συνόλου

Εμείς υλοποιούμε την calculate_precision συνάρτηση, η οποία υπολογίζει την θεωρητική τομή των τροποποιημένων σε set retrieved και relevant, παίρνει το πλήθος των πραγματικά relevant από αυτά που βρήκαμε και το διαιρούμε με το πλήθος του set των retrieved, υπολογίζοντας έτσι την ακρίβεια για δύο σύνολα εγγράφων.

---

2. Ανάκληση - Recall

Η ανάκληση απαντά στο ερώτημα του “από όλα τα έγγραφα που ξέρουμε ότι είναι σχετικά στη συλλογή μας, πόσα από αυτά βρήκε το σύστημα μας;”.

Recall = $\frac{|\,Retrieved \cap Relevant\,|}{|\,Relevant\,|}$, όπου | . | ο πληθάριθμος του συνόλου

Στην υλοποίηση μας, ακολουθήσαμε την ίδια ακριβώς πρακτική με το precision όμως επειδή ψάχνουμε πόσα relevant βρήκαμε από όλα τα γνωστά relevant docs και όχι πόσα relevant βρήκαμε σε όσα ανακτήσαμε, διαιρούμε με το μέγεθος του συνόλου των Relevant γιατί αυτό είναι το ενδιαφέρον πεδίο στην προκειμένη.

---

3. F1-Score - Αρμονικός Μέσος Όρος

Επειδή η ακρίβεια και η ανάκληση είναι μερικώς αντιστρόφως ανάλογα (π.χ. προσπαθώντας να βρώ ψηλή ανάκληση(τα πάντα), φέρνω χαμηλή ακρίβεια (πράγματα που δεν θέλω)), το F1-score είναι ο αρμονικός Μ.Ο. των δύο (ισορροπία του συστήματος).

F1 = $\frac{2}{\frac{1}{R}+\frac{1}{P}}$ = 2 x $\frac{P\,x\,R}{P+R}$, όπου R, P η ανάκληση και η ακρίβεια του συστήματος. Η υλοποίηση μας ακολουθεί τον παραπάνω μαθηματικό τύπο.

---

4. Precision@k - Ακρίβεια στα k docs

Επειδή ο χρήστης συνήθως βλέπει μόνο τα πρώτα αποτελέσματα, πρέπει να δούμε την ακρίβεια των πρώτων k αποτελεσμάτων που επέστρεψε το σύστημα μας.

Η υλοποίηση μας υπολογίζει την ακρίβεια αλλά για την κομμένη λίστα όλων των αποτελεσμάτων που επέστρεψε το σύστημα μας στα πρώτα k από αυτά (retrieved_ids[:k]).
Έτσι στην νέα λίστα (sliced λίστα όλων των retrieved) καλεί την calculate_precision, υπολογίζοντας έτσι την ακρίβεια σαν να είχε επιστρέψει εξαρχής μόνο k έγγραφα.

$$P@k = \frac{\#\{relevant\ docs\ in\ top\ k\ results\}}{k}$$

---

5. Διάγραμμα Ανάκλησης-Ακρίβειας - Precision-Recall Curve

Η γραφική παράσταση αυτή δείχνει τη σχέση μεταξύ ακρίβειας και ανάκλησης. Ιδανικά θα θέλαμε να έχουμε και υψηλή ακρίβεια και υψηλή ανάκληση, κάτι που δυστυχώς δεν γίνεται και έτσι παρατηρούμε ένα trade-off των δύο (όσο ανεβαίνει το ένα πέφτει το άλλο).

Με τη χρήση της matplotlib λοιπόν, σαρώνουμε την λίστα των αποτελεσμάτων αυξάνοντας το k από 1 μέχρι και το τελευταίο έγγραφο, υπολογίζοντας το ζεύγος Precision-Recall σε κάθε βήμα (k), τα αποθηκεύουμε και τα σχεδιάζουμε όπως φαίνονται εδώ.

Ερώτημα 4 - Συγκρίσεις

Στο ερώτημα αυτό, για να βρούμε τις βέλτιστες παραμέτρους του TfidfVectorizer, εφαρμόζουμε μια συστηματική διαδικασία κατά την οποία δημιουργούμε 36 συνολικά συνδυασμούς παραμέτρων και εκτελούμε το πείραμα ανάκτησης για κάθε έναν ξεχωριστά.

Για την ακρίβεια διαλέξαμε αυτό το σύνολο υπερπαραμέτρων του TfidfVectorizer:

- **ngram_range: (1, 1), (1, 2)** : Unigrams vs Bigrams, καθορίζει αν κοιτάμε τις λέξεις μεμονωμένες ή σε ζεύγη.
- **min_df & max_df**: αγνοεί λέξεις που εμφανίζονται ελάχιστες φορές (όπως τυπογραφικά) και αντίστοιχα αγνοεί λέξεις που εμφανίζονται πολύ στα έγγραφα (όπως stopwords).
- **sublinear_tf**: χρήση λογαριθμικής στάθμισης (1 + log(tf)) στον όρο tf ή και όχι για να μειώσουμε την ισχύ των λέξεων που επαναλαμβάνονται πολλές φορές στο ίδιο κείμενο.
- **norm**: εδώ κρατήσαμε σταθερά την l2 νόρμα για να κανονικοποιήσουμε τα μεγάλα κείμενα έτσι ώστε να συγκρίνονται ισότιμα με τα μικρά.

Κριτήρια επιλογής βέλτιστου μοντέλου

Για την επιλογή του καλύτερου συνδυασμού βασιστήκαμε ιεραρχικά σε δύο μετρικές:

1. **MAP**(Mean Average Precision):
   το πρωτεύον κριτήριο. Το MAP δείχνει τη γενική ευστάθεια του συστήματος λαμβάνοντας υπόψη τη σειρά κατάταξης όλων των σχετικών εγγράφων. Το MAP μας πληροφορεί όχι απλά για το πόσα σωστά βρήκαμε, αλλά και πόσο ψηλά στη λίστα τα βάλαμε. Πρώτα υπολογίζουμε το Average Precision (AP) για κάθε ερώτημα ξεχωριστά και μετά βγάζουμε τον μέσο όρο για όλα.

   - AP = $\frac{1}{|R|}\, x \sum_{k=1}^{n} (P(k)\, x\, rel(k))$, όπου |R| ο αριθμός όλων των σχετικών με το ερώτημα εγγράφων, P(k) η ακρίβεια στη θέση k και rel(k) δείκτης που είναι 1 αν το έγγραφο είναι σχετικό και 0 αν όχι.
   - Mean AP:

     MAP = $\frac{1}{|Q|}\, x \sum_{q=1}^{|Q|} AP(q)$, δηλαδή ο απλός Μ.Ο. για όλα τα queries

2. **P@10**:
   χρησιμοποιήθηκε σε περιπτώσεις ισοβαθμίας του MAP.

Στο τελικό στάδιο συγκρίναμε 3 διαφορετικά συστήματα:

1. Optimized Sklearn TfidfVectorizer: το μοντέλο της βιβλιοθήκης αφού βρήκαμε τις βέλτιστες παραμέτρους με τα παραπάνω κριτήρια
2. Custom Classic: το VSM μας με απλό TF-IDF
3. Custom Alternative: το VSM μας με alternative tf-idf το logarithmic-tf (1+log(tf))

και καταγράψαμε τους χρόνους indexing (πόσο χρόνο θέλει το σύστημα για να διαβάσει τη συλλογή και να χτίσει το ευρετήριο) και search (πόσο χρόνο θέλει να απαντήσει σε όλα τα ερωτήματα).

Ερώτημα 5 - Clustering με TF-IDF και προεκπαιδευμένες ενσωματώσεις

Στο ερώτημα αυτό σκοπός μας ήταν η ομαδοποίηση-clustering των εγγράφων σε διακριτές ομάδες. Εστιάσαμε λοιπόν σε δύο μορφές αναπαράστασης των δεδομένων για να δούμε πως επηρεάζεται η ποιότητα των clusters.

1. **Λεξιλογική αναπαράσταση** TF-IDF (Ακριβή ταύτιση) μέσω χρήση του βελτιστοποιημένου TfidfVectorizer από το προηγούμενο ερώτημα:

   Καθαρίζουμε τα terms, φτιάχνουμε το inverted index μετρώντας τις επαναλήψεις του όρου σε κάθε έγγραφο και εφαρμόζουμε τα βάρη κάθε όρου στα διανύσματα φτιάχνοντας έτσι ένα μητρώο με γραμμές τα docs και στήλες τα terms με τιμή doc-term το αντίστοιχο weight. Βασίζεται σε ακριβή ταύτιση λέξεων, αν δύο docs έχουν κοινές λέξεις τότε θεωρούνται όμοια. Όμως, δεν έχουμε συνώνυμα και σύνδεση μεταξύ λέξεων.

2. **BERT embeddings** (Σημασιολογική αναπαράσταση) με χρήση το προεκπαιδευμένο μοντέλο all-MiniLM-L6-v2 από την βιβλιοθήκη sentence-transformers:

   Το μοντέλο αυτό είναι ένα συμπυκνωμένο version του BERT της google αλλά κάνοντας χρήση των μισών διαστάσεων (διανύσματα μεγέθους 384 αντί για 768), αυξάνοντας έτσι την απόδοση. Στην ουσία, το μοντέλο έχει για κάθε λέξη έναν vector σε μια βιβλιοθήκη ήδη συνδυασμένο με κάποιους όρους σε κάθε διάσταση (π.χ. για το Apple έχει συνιστώσα “Fruit” και “Tech Company”) και με βάση τις λέξεις που περιέχονται στο ίδιο έγγραφο (το οποίο διαβάζει μια μπρός και μια πίσω / Self- Attention), αυξομειώνει τις συνιστώσες συσχέτισης της λέξεις με τις άλλες. Έπειτα κάθε vector της λέξης του εγγράφου μεγέθους 384 συμπτύσσεται σε ένα μοναδικό vector που αντιπροσωπεύει το έγγραφο πλέον μεγέθους 384. Άρα έχουμε ένα μητρώο με μεγέθους #docs x 384 το οποίο πλέον έχει για κάθε γραμμή (έγγραφο) τις συσχετίσεις των λέξεων μεταξύ τους.

   Για αυτά τα δύο διαφορετικά μητρώα πάμε και βρίσκουμε τον βέλτιστο αριθμό του πλήθους των clusters (k). Μέσω της μετρικής Silhouette Score (πόσο μοιάζει ένα έγγραφο με τα άλλα έγγραφα της δικής του ομάδας σε σύγκριση με τα έγγραφα των άλλων ομάδων) τρέχουμε τον αλγόριθμο για k = 2 μέχρι 8 και επιλέγουμε το k που μεγιστοποιεί αυτό το σκόρ.

   Έπειτα αφού βρούμε το optimal k , εκτελούμε τον αλγόριθμο KMeans διαλέγοντας k random cluster centroid, υπολογίζοντας τις ομάδες/clusters και βρίσκουμε τον median αυτών θέτοντας το σαν το νέο centroid και ξανά το ίδιο μέχρι τα κέντρα και οι ομάδες να μην αλλάζουν.

Έπειτα αφού έχουμε υπολογίζει όλα τα clusters, επειδή είναι αδύνατο να δούμε γραφική παράσταση σε τόσες πολλές διαστάσεις (χιλιάδες για το ένα και 384 για το άλλο), χρησιμοποιούμε την τεχνική PCA (Principal Component Analysis, συμπιέζει τις διαστάσεις κρατώντας τις σημαντικές πληροφορίες που διαφοροποιούν τα δεδομένα) για να τα απεικονίσουμε σε 2D χώρο όπως φαίνονται εδώ.

Αποτελέσματα και Παρατηρήσεις:

Πειραματικό πλαίσιο:

Παρακάτω παρουσιάζουμε τις απαραίτητες γραφικές και σχόλια για την αξιολόγηση και περιγραφή των αποτελεσμάτων των διαδικασιών των απαιτήσεων της εργασίας. Υλοποιώντας τις διαδικασίες παράξαμε αποτελέσματα ενδεικτικά σε εικόνες και comparative γραφικές προκειμένου να προσδιορίσουμε τα πλεονεκτήματα και μειονεκτήματα, τα αποτελέσματα και τη συμπεριφορά κάθε μοντέλου σε κάθε συνιστώμενο απο τα ερωτήματα περιβάλλον. Περισσότερα σχόλια μας από τη θεωρία αλλα και υλοποίηση των διαδικασιών βρίσκονται στην περιγραφή της υλοποίησης.

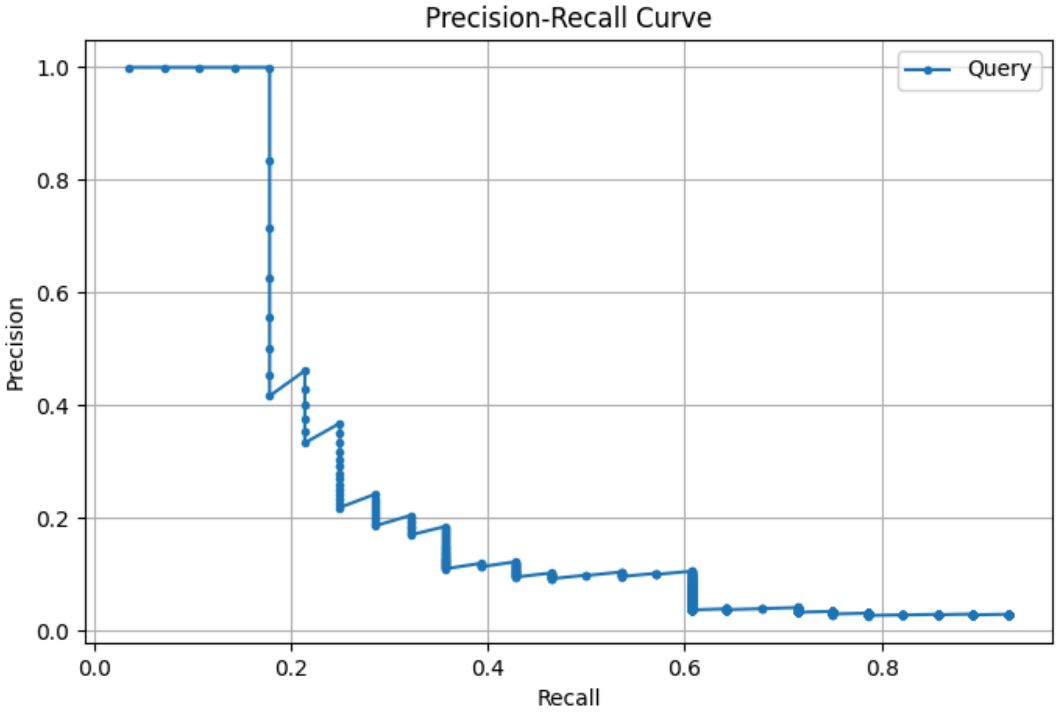
Ερωτήματα 1:4 - Μετρικές και Αξιολόγηση

Όσον αφορά τα πορισματα μας κατα την διεξαγωγή πειραμάτων και αποτελεσμάτων των υλοποιήσεων μας, θα μπορούσαμε να έχουμε βελτιώσει τους χρόνους αναζήτησης έχοντας ζυγίσει κατάλληλα το vsm structure ανεξαρτήτως του query (κατα τη κατασκευή του). Οι μετρήσεις είναι ικανοποιητικές απέναντι στα statistics του TfIdfVectorizer ειδικά δεδομένου του πολύ trivial preprocessing που δεν περιέχει complicated stemming και καθόλου lemmatization. Ειδικά η πιο informed logarithmic τεχνική ζυγίσματος μιμείται πολυ καλα την καμπύλη του ιδανικού TfIdfVectorizer, γεγονός που αποδίδουμε στην αποδοτικότερη αξιολόγηση των components κατα τη σύγκριση (όπως φαίνεται και από τα αντίστοιχα map values). Επίσης αξιοσημείωτο είναι το γεγονός πως το σύστημα διατηρεί μεγάλο precision για πολλές τιμές του recall μέχρι που τελικά εμπίπτει στο tradeoff σε όλα τα σενάρια υλοποίησης.

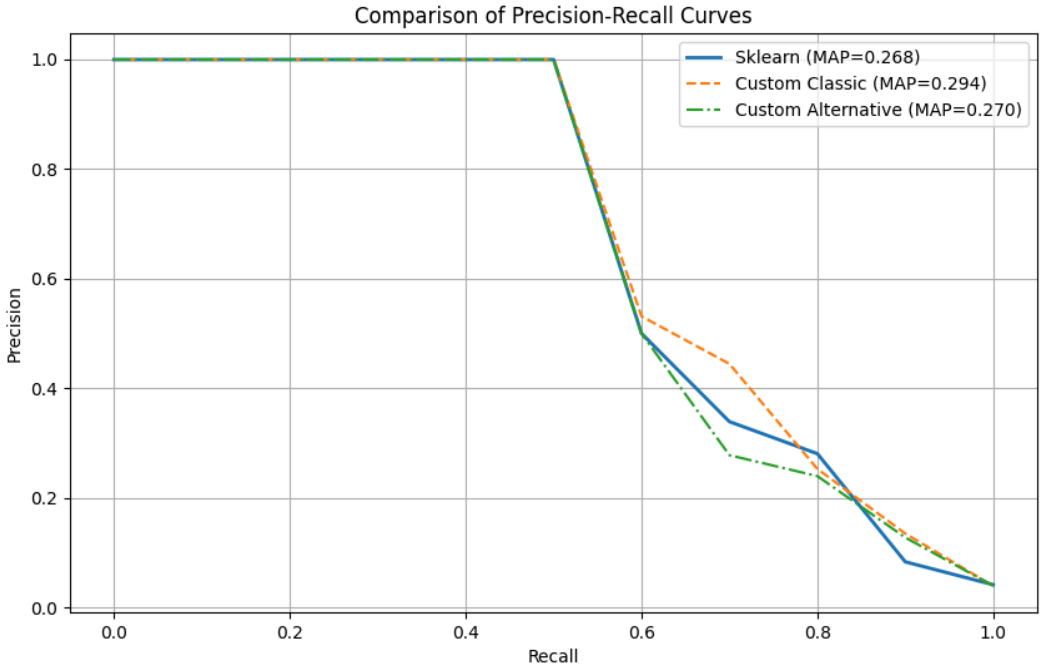
Χρόνοι για Indexing και Searching

Model	Idx(s)	Srch(s)	MAP	P@10	F1
Optimized Sklearn	0.0381	0.0043	0.2678	0.4150	0.0702
Custom VSM (Classic)	0.0262	33.4915	0.2936	0.4550	0.0676
Custom VSM (Alternative)	0.0262	33.4798	0.2697	0.4300	0.0676

VSM Plots and Comparisons

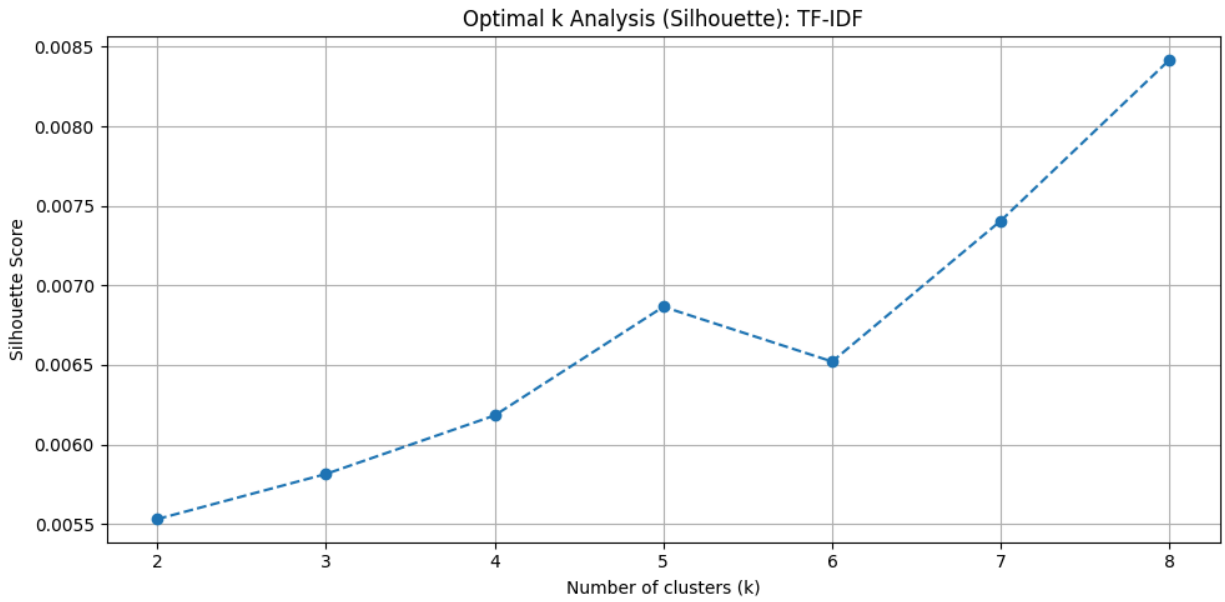


Σχήμα γραφικής precision vs recall για το inverted index vsm search

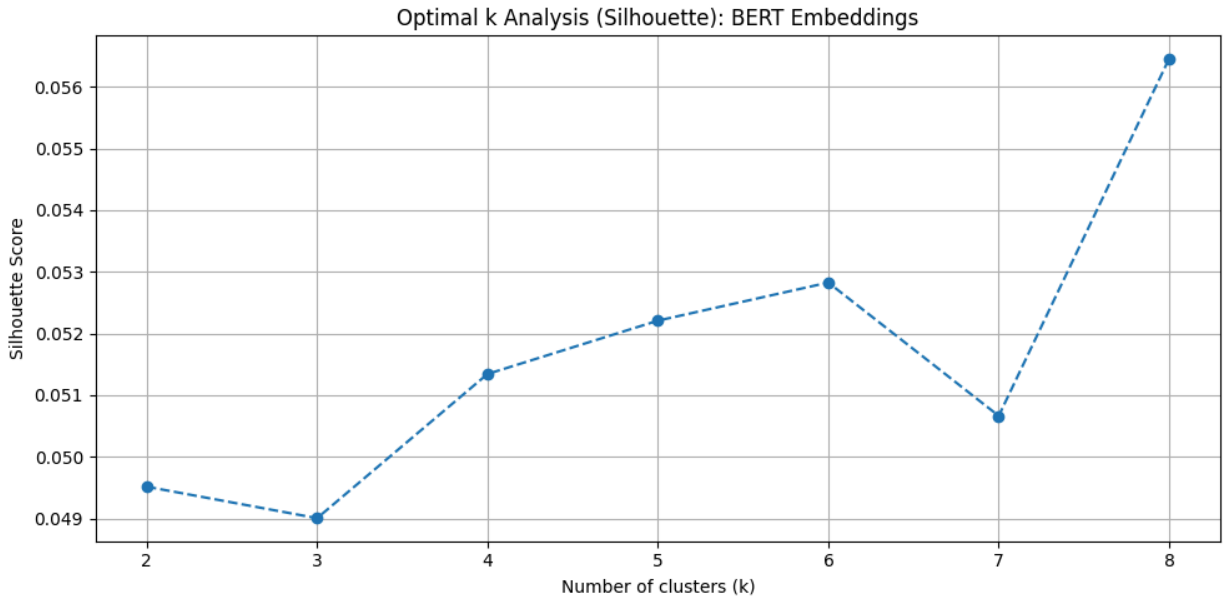


Σχήμα γραφικής precision vs recall για Idf Vectorizer implementation, Custom Classic: TF-IDF VSM και Custom Alternative: logarithmic TF-IDF

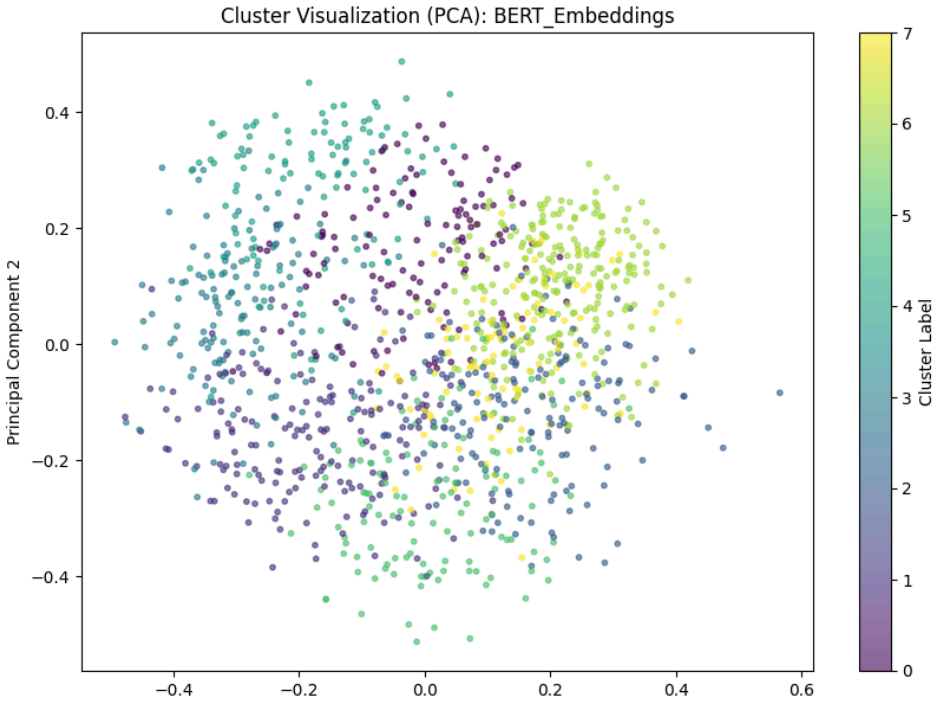
Clustering Plots and Comparisons

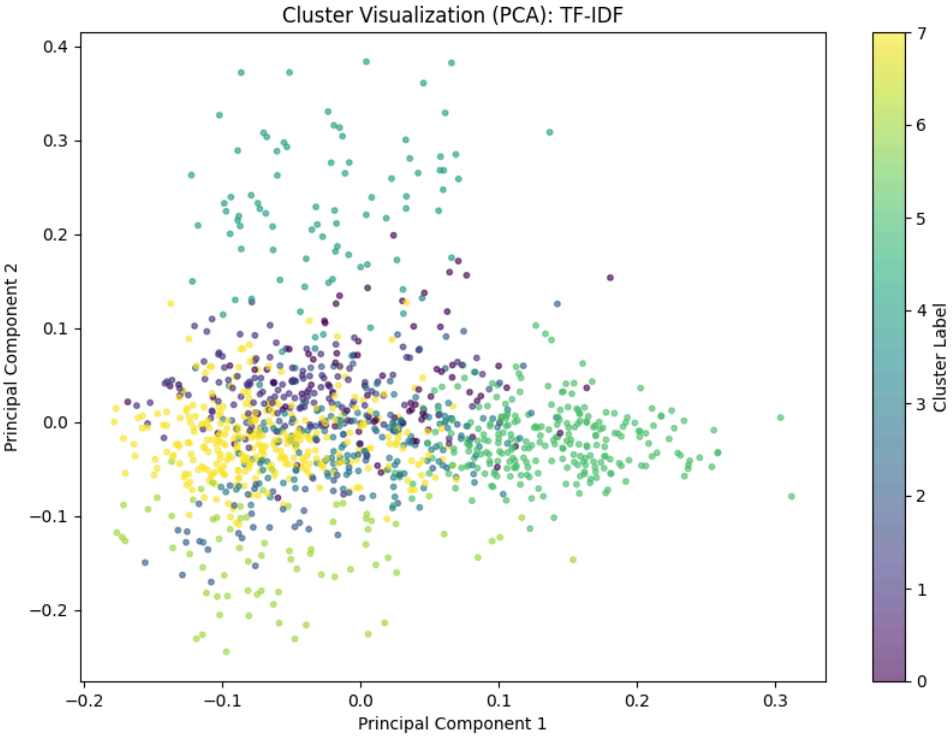


Σχήμα για έρευνα βελτίστου κ στο Vectorizer παράγων μέσω silhouette score



Σχήμα για έρευνα βελτίστου κ στο BERT παράγων μέσω silhouette score


Cluster Visualization (PCA): BERT_Embeddings
Principal Component 2
Cluster Label
0.4
0.2
0.0
−0.2
−0.4
−0.4
−0.2
0.0
0.2
0.4
0.6
7
6
5
4
3
2
1
0


Cluster Visualization (PCA): TF-IDF
Principal Component 2
Principal Component 1
Cluster Label
0.4
0.3
0.2
0.1
0.0
−0.1
−0.2
−0.2
−0.1
0.0
0.1
0.2
0.3
7
6
5
4
3
2
1
0

Αναφορές - Βιβλιογραφία

[2] Disproportionate Scaling in Normalization and effect on scale sensitive methods
Investigating the Impact of Min-Max Data Normalization on the Regression Performance of K-Nearest Neighbor with Different Similarity Measurements - Peshawa J. Muhammad Ali - Koya University

Book reference to Inverted Index logic for IR
Introduction to Modern Information Retrieval, Gerard Salton, Michael J. McGill

Paper reference to VSM utilization in IR Modeling
Matrices, Vector Spaces, andInformation Retrieval - Michael W. Berry, Zlatko Drmac Elizabeth R. Jessup

Παράρτημα

Οποιοδήποτε σχόλιο ή παρατήρηση, πόρισμα και σημείωση έχουν γίνει παραπάνω στα αντίστοιχα σημεία τεκμηρίωσης της υλοποίησης η αξιολόγησης των αποτελεσμάτων.